

An MLIR Lowering Pipeline for Stencils at Wafer-Scale

Nicolai Stawinoga^{*†}

Technische Universität Berlin
Berlin, Germany
nicolai@aes.tu-berlin.de

David Katz^{*}

EPCC
University of Edinburgh
Edinburgh, United Kingdom
d.kacs@sms.ed.ac.uk

Anton Lydike

School of Informatics
University of Edinburgh
Edinburgh, United Kingdom
anton.lydike@ed.ac.uk

Justs Zarins

EPCC
University of Edinburgh
Edinburgh, United Kingdom
j.zarins@epcc.ed.ac.uk

Nick Brown

EPCC
University of Edinburgh
Edinburgh, United Kingdom
nick.brown@ed.ac.uk

George Bisbas[†]

Imperial College London
London, United Kingdom
georgios.a.bisbas@gmail.com

Tobias Grosser

University of Cambridge
Cambridge, United Kingdom
tobias.grosser@cst.cam.ac.uk

Abstract

The Cerebras Wafer-Scale Engine (WSE) delivers performance at an unprecedented scale of over 900,000 compute units, all connected via a single-wafer on-chip interconnect. Initially designed for AI, the WSE architecture is also well-suited for High Performance Computing (HPC). However, its distributed asynchronous programming model diverges significantly from the simple sequential or bulk-synchronous programs that one would typically derive for a given mathematical program description. Targeting the WSE requires a bespoke re-implementation when porting existing code. The absence of WSE support in compilers such as MLIR, meant that there was little hope for automating this process.

Stencils are ubiquitous in HPC, and in this paper we explore the hypothesis that domain specific information about

stencils can be leveraged by the compiler to automatically target the WSE without requiring application-level code changes. We present a compiler pipeline that transforms stencil-based kernels into highly optimized CSL code for the WSE, bridging the semantic gap between the mathematical representation of the problem and the WSE's asynchronous execution model. Based upon five benchmarks across three HPC programming technologies, running on both the Cerebras WSE2 and WSE3, our approach delivers comparable, if not slightly better, performance than manually optimized code. Furthermore, without requiring any application level code changes, performance on the WSE3 is around 14× faster than 128 Nvidia A100 GPUs and 20× faster than 128 nodes of a CPU-based Cray-EX supercomputer when using our approach.

^{*}Both authors contributed equally to this work

[†]Affiliation at the time of contribution

Authors' Contact Information: Nicolai Stawinoga, Technische Universität Berlin, Berlin, Germany, nicolai@aes.tu-berlin.de; David Katz, EPCC, University of Edinburgh, Edinburgh, United Kingdom, d.kacs@sms.ed.ac.uk; Anton Lydike, School of Informatics, University of Edinburgh, Edinburgh, United Kingdom, anton.lydike@ed.ac.uk; Justs Zarins, EPCC, University of Edinburgh, Edinburgh, United Kingdom, j.zarins@epcc.ed.ac.uk; Nick Brown, EPCC, University of Edinburgh, Edinburgh, United Kingdom, nick.brown@ed.ac.uk; George Bisbas, Imperial College London, London, United Kingdom, georgios.a.bisbas@gmail.com; Tobias Grosser, University of Cambridge, Cambridge, United Kingdom, tobias.grosser@cst.cam.ac.uk.



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '26, Pittsburgh, PA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2359-9/2026/03

<https://doi.org/10.1145/3779212.3790124>

CCS Concepts: • **Computer systems organization** → Multiple instruction, multiple data; Systolic arrays; Multicore architectures; • **Computing methodologies** → *Parallel programming languages*; • **Software and its engineering** → **Domain specific languages**.

Keywords: Cerebras, Wafer Scale Engine, domain-specific languages, intermediate representations, stencil computations, MLIR, xDSL

ACM Reference Format:

Nicolai Stawinoga, David Katz, Anton Lydike, Justs Zarins, Nick Brown, George Bisbas, and Tobias Grosser. 2026. An MLIR Lowering Pipeline for Stencils at Wafer-Scale. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26)*, March 22–26, 2026, Pittsburgh, PA, USA. ACM, New York, NY, USA, Article 1, 16 pages. <https://doi.org/10.1145/3779212.3790124>

1 Introduction

The Cerebras **Wafer-Scale Engine (WSE)** [26] is a *systolic array* [22] spanning a full wafer of 900,000 independent interconnected **Processing Elements (PEs)** on the **WSE3**. Originally designed as a novel class of AI accelerator, and now in its third architectural generation, the Cerebras **WSE** has been applied to a variety of impactful HPC applications [20, 28, 32, 33, 41, 44, 45, 47, 49]. Indeed Ltaief et al. [28] demonstrated that due to the large amount of distributed SRAM, the WSE is capable of delivering performance where other architectures hit memory-bound bottlenecks. Driven by the potential for very high performance, there is significant interest in using the WSE for HPC workloads, however very few people are actually using the architecture and this is due to the major blocker being the effort required to port codes.

Although a significant time is required to rewrite codes in Cerebras’s bespoke general-purpose CSL programming language, the programmer’s first challenge is the algorithmic transformation required to suit the WSE. This recasting of an algorithm requires substantial time and expertise on behalf of the programmer, often involving architecture-specific techniques. Consequently, whilst there have been several heroic efforts to bring simple kernels or benchmarks to the WSE, the technology is yet to enjoy widespread use or adoption in HPC due to these programming challenges.

In this paper we present a compiler flow, based on MLIR, that enables codes to run on the WSE without modification. Our overarching research hypothesis is that it is possible to solve the fundamental programmability and performance challenges on esoteric architectures, such as the WSE, by using compiler automation driven by domain specific abstractions present in user code. Targeting three front-ends across two scientific domains our proposition is that software tooling, such as MLIR, that was initially designed with AI workloads in mind can be leveraged to solve some of the fundamental challenges around exploiting AI-based hardware for High Performance Computing (HPC).

The contributions of this paper are:

- A discussion of the WSE’s highly asynchronous execution model and how it affects program representation when targeted by code generation (Section 2).
- A set of intermediate domain-specific and domain-agnostic dialects to convert stencil expressions to CSL code via our low-level `cs1-ir` dialect that re-implements a large subset of CSL (Section 4).
- A strategy of lowering transformations to generate the `cs1-ir` dialect from higher level MLIR abstractions (Section 5).
- An evaluation of our stencil lowering pipeline for the WSE2 and WSE3, ultimately demonstrating that our approach delivers slightly better performance than hand-written code supporting multiple generations of

the WSE – likely the first performance comparison of the WSE2 and WSE3 at the time of writing (Section 6).

2 Motivation: The Cerebras WSE and its asynchronous execution model

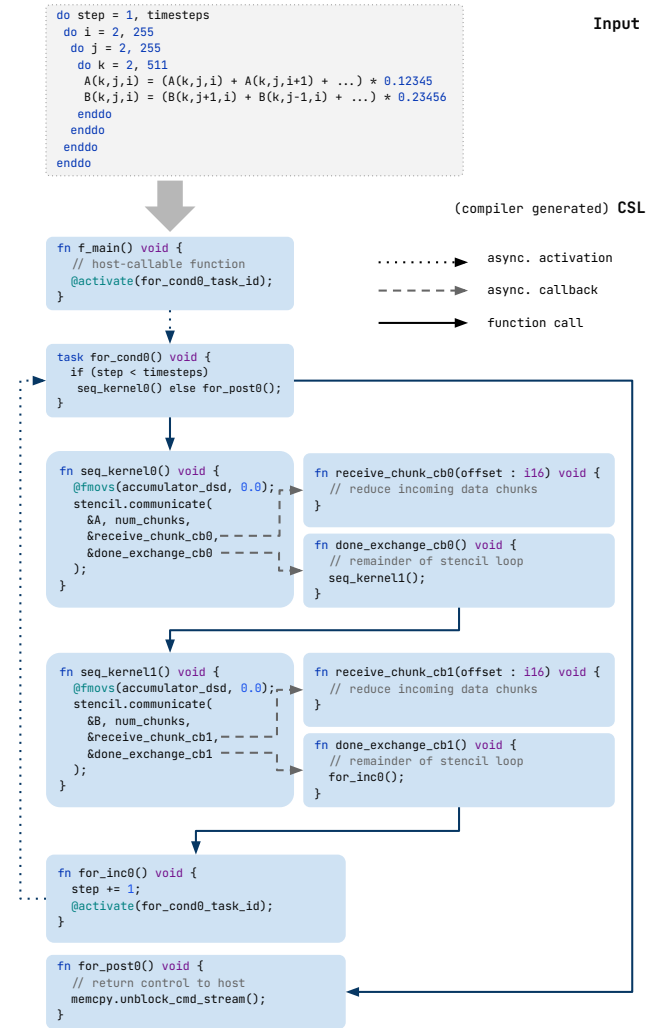


Figure 1. A time-step loop surrounding asynchronous data exchanges in CSL.

The Cerebras Wafer-Scale Engine (WSE) features a unique and highly-asynchronous execution model characterized by its compute architecture. It features a systolic array of up to 900,000 Processing Elements (PEs), where each comprises a computational core, router, program counter, clock, and 48KB of SRAM memory. PEs can be programmed individually and act fully independently of one another, managing their own memory to which they are able to perform a 128-bit memory read and a 64-bit memory write in each cycle. The impressive total 40GB (44GB) of on-chip memory for the WSE2 (WSE3) architecture leveraged in this work does not

form a unified memory system, and no caches or memory hierarchy exists.

Data between PEs is shared explicitly via four bi-directional communication channels, connecting the routers of PEs to that of their immediate cardinal north, south, east, and west neighbors. An additional channel, called *the ramp*, connects the router to the attached core. A 32-bit message, known as a *wavelet*, can be sent between neighboring PEs in a single clock cycle in each direction, resulting in an aggregate peak fabric bandwidth of 214 Pb/s on the WSE3. Twenty-four virtual channels, known as *colors*, provide an abstraction to the programmer for configuring routes and switches. When an asynchronous data transfer has completed, the PE can flag an internally bound *task* as unblocked or as ready for activation.

In 2022 Cerebras released their Cerebras Software Language (CSL) SDK which enables direct programming of the WSE’s architecture. CSL is syntactically based on Zig, featuring numerous extensions to explicitly support the WSE, such as tasks, wavelets, colors and Data Structure Descriptors (DSDs) which enable performing operations across arrays for efficiency. However, the exposing of the WSE’s execution model in practice makes CSL programming appear fundamentally different from programming in Zig. Besides in-depth knowledge of the architecture, programmers must also fundamentally recast their algorithms moving from a form suitable to CPUs or GPUs to one fitting the WSE.

Figure 1 illustrates the representation of a time-step loop surrounding a naive Jacobian stencil program in Fortran (top) and CSL (bottom). The assumption is that the two stencil operations contained in the time-step require asynchronous communication between PEs. Therefore, although CSL provides various loop constructs, it is not possible to represent this program in a top-level loop, as CSL lacks a mechanism to achieve re-synchronization within a code block (e.g., *await/barrier* in single-threaded and multi-threaded contexts, respectively). Consequently, any control flow that surrounds asynchronous operations such as communication must be translated into tasks and callbacks. The programmer must therefore decompose the program into separate tasks which are then scheduled for execution based on data arrival. In languages designed for CPUs and conventional accelerators, an outer loop such as *do step = 1, timesteps* is a simple language construct, however on the WSE this same algorithm must be recast as tasks driven by callbacks, due the communication required between iterations of the outer loop.

The call to `stencil.communicate()` in `seq_kernel0()` and `seq_kernel1()` of Figure 1 encapsulates boiler-plate code for chunk-wise data exchanges and internally uses additional tasks. Practicalities of the architecture, specifically the limited amount of memory per PE, severely limit the size of buffers and-so data must be exchanged in chunks. Consequently, this function accepts two user-provided callbacks

(triggered indirectly as indicated by dashed arrows); the first that is triggered to process each incoming chunk of data, and the second is executed once the entire exchange of data has completed. The algorithm’s overall execution flow of is then continued from the second callback, and this demonstrates the complexity faced by the programmer when they wish to port even a seemingly simple kernel to the WSE because CSL directly exposes low-level architectural concepts, making the programming complex and susceptible to errors.

2.1 An actor-based asynchronous execution model

The semantic gap in program representations depicted in Figure 1 directly follows differences in the execution model of the underlying architecture. Unlike conventional data-parallel accelerators, the WSE’s is primarily a task-parallel architecture in a manner resembling the *actor* paradigm. Originally published under the title *A universal modular ACTOR formalism for artificial intelligence* in 1973 [17] (over 50 years ago), the formalism predates hardware and software implementations, yet well describes the execution model of the WSE both on a hardware and software level.

On the hardware level, we conceive of *PEs as hardware actors* in a static arrangement. A key characteristic of the actor model is that data is passed as messages between actors and on the WSE each PE contains a set of queues, acting as mailboxes, into which data is received. Actors also hold their own state, and on the WSE each PE acts independently and contains its own clock and private memory. There are two types of messages, or *wavelets* that can be passed between hardware actors, *data wavelets* that contain a data payload, and *control wavelets* to instruct another actor to update its routing configuration. Depending on the PEs state messages are forwarded or consumed, which in some cases involves activating a software actor. PEs as hardware actors have a physical location, a concept not shared by software implementations of the actor model, and whilst there is library support to pass messages between arbitrary PEs, exploiting spatial proximity for neighbor to neighbor communication is crucial for performance in practice.

On a software level, we conceive of *tasks as software actors*. The code running inside each PE provides a software implementation of the actor model via tasks. There are three types of tasks provided by CSL, each of which with hardware support:

- `data_tasks` are message listeners triggered when receiving a message, or *data wavelet*, in the form of data on the queue they are configured to listen to.
- `control_tasks` are similar to data tasks but are triggered by *control wavelets*, messages that do not carry a data payload.
- `local_tasks` can only be triggered internally and are particularly useful as callbacks when completing asynchronous sending or receiving of data.

As program state is maintained by the PE, tasks have a shared program state, although separate state could theoretically be enforced by the programmer. Tasks as software actors do not exhibit true parallelism, as each PE is strictly single-threaded, but instead tasks provide an asynchronous execution model. Passing execution flow between tasks adds significant complexity, and is frequently referred to as ‘*callback hell*’ in literature [12]. The major challenge is that CSL exposes this problem to the programmer without providing higher-level language abstractions, such as *futures* [10], that could help address it. Re-structuring a program from its synchronous representation to the asynchronous model creates the *continuation complexity problem* [48] of passing execution flow, and this was a challenge we had to address in our IR transformation as described in Section 5.

To summarize, much of the complexity involved in programming the WSE is due to having to map synchronous algorithms to the asynchronous nature of the WSE. It is important to note, however, that CSL is not formally an actor-based programming language, exposing the user to problems known from asynchronous actor-like programming, including the complexity of managing interactions in barrier-free parallelism, preventing race conditions, and more.

2.2 Departing from classic dataflow architectures

Whilst the WSE is a dataflow architecture, the coarse grained nature of the design means that it is very different from other, fine grained, dataflow architectures such as FPGAs and this provides unique challenges for distributed stencil computations and compilation. Unlike single units of logic that stream data in fine-grained dataflow architectures, each PE in the WSE contains a vectorized CPU that runs tasks and is capable of a much greater level of compute. Consequently, the pipelined approach to parallelism that is natural on fine grained architectures such as FPGAs is not applicable on the WSE, as workloads comprise both local and remote compute. Indeed, on the WSE the programmer must adopt a combination of parallelism and optimization techniques across both dataflow (e.g. communication between PEs involves streaming many individual pieces of data) and distributed memory (e.g. for stencils decomposing geometrically across PEs) architectures and this hybrid approach is very difficult.

As an example, leveraging shift buffers [11] as bespoke memory on FPGAs to implement highly efficient stencil computations is a well known technique [6]. However the nature of the WSE means that this approach is neither possible, as memory is local and private to PEs, nor optimal, as one must carefully design inter-PE communications. Instead, for efficient execution on the WSE one must adopt a different approach to decomposing their problem across this dataflow architecture and, as described in this section leveraging asynchronous tasks that interleave local computation and handling the arrival of remote data.

3 Compiler ecosystems and stencils

MLIR. MLIR [24] provides a set of reusable IR dialects and transformations between them; separate dialects can be mixed into the IR and manipulated independently. Similar to the more low-level LLVM-IR [23], MLIR follows the Static Single Assignment (SSA) form. Reusing existing dialects and transformations provides much greater sharing of compiler infrastructure, significantly reducing the overall software effort in developing new compilers. Furthermore, MLIR is also a framework where compiler developers can create their own dialects and transformations. The pluggable nature of the framework allows the creation of deep compile chains with a wide range of reusable dialects, backends, or frontends.

xDSL. MLIR imposes a steep learning curve on developers where they must, for example, understand LLVM concepts, work with the Tablegen format to describe dialects, and keep track of the fast-evolving MLIR repository. xDSL was developed as a Python-based compiler design toolkit that is 1-1 compatible with MLIR. Dialects are expressed in the IRDL [13] format within Python classes, enabling rapid exploration and prototyping of MLIR concepts, with a more mature version then committed into the MLIR codebase once proven. The MPI dialect is one example of this approach, where it was first developed in xDSL [4] before being standardized into MLIR. As xDSL is 1-1 compatible with MLIR, the IR can freely pass between the two frameworks during compilation. We used xDSL to develop the transformations and optimizations described in this paper, and due to xDSL being based upon MLIR, we use the two terms interchangeably unless otherwise stated.

Listing 1 Sketch of Fortran stencil code example which adds values from two grid cells and multiplies result by a constant

```

1  do i = 2, 255
2    do j = 2, 255
3      do k = 2, 511
4        data(k,j,i) = (data(k,j,i) + data(k,j,i+1)) * 0.12345
5      enddo
6    enddo
7  enddo
```

The stencil dialect. The stencil dialect was initially developed as part of the Open Earth Compiler [15] and is now part of xDSL. A stencil is a geometric arrangement of a group of neighboring grid cells that, by using a numerical approximation routine, relate to a specific grid cell of interest. Listing 1 provides an example of a Fortran stencil code which multiplies the result of adding values from two grid cells by a constant, and Listing 2 (see Section 5) sketches the corresponding SSA-based MLIR using the `stencil` and `arith` dialects to express this calculation. The nested loops at lines 1, 2 and 3 of Listing 1 have been transformed into the `stencil.apply`

operation in the IR, where the `stencil.access` operations in Listing 2 correspond to accesses on the `data` array at line 4 of the Fortran code. Within the `stencil.apply` operation the stencil dialect determines data accesses, and the `arith` dialect determines the computation. It should be noted that the `stencil.apply` operator is running across the entire grid, whose lower and upper bounds are determined by the types of the input and output fields, executing the body of the `stencil.apply` operation for every grid cell.

Stencil generation from PSyclone, Devito & Flang. PSyclone [1] is a climate and weather DSL enabling scientists to write their kernels in Fortran, while abstracting away the low-level details of parallelism. It is used in the Met Office’s next-generation weather and ocean models [30, 34]. The PSyclone compiler analyzes the scientist’s code and undertakes optimizations and transformations upon an internal IR, such as applying OpenMP or OpenACC at the loop level for threaded parallelism and GPU acceleration respectively.

Devito [5, 27, 29], is an open-source Python-based DSL and compiler framework widely used in academia and industry. Based around a symbolic DSL in Sympy [31], its purpose is to ease the development of HPC Finite-Difference (FD) PDE solvers. Devito is used in a range of industries, especially those focused on seismic and medical workloads.

It was highlighted in Bisbas et al. [4] that whilst these two DSLs appear rather different, they share a lot of common ground for compiler optimizations. Consequently, Bisbas et al. [4] integrated PSyclone and Devito with the stencil dialect via xDSL, the objective being to share compiler infrastructure. It was observed that by being in the MLIR ecosystem both DSLs benefited from optimization passes developed by the wider community, including the vendors, and indeed this integration resulted in performance benefits on CPUs and GPUs. These optimizations are further discussed in Section 5.7. This motivated integration of the stencil dialect with the Flang Fortran compiler [9], which itself is built atop MLIR. Performance delivered by Flang fell significantly short of other compilers [7], and by the addition of a compiler pass identifying and extracting stencils within Fortran code, it was demonstrated that one could close this gap [9].

It is these three frontends, PSyclone, Devito, and Flang, that we leverage in this work and use as benchmarks in Section 6. Indeed, Rodriguez-Canal et al. developed a lowering from the stencil dialect to an efficient FPGA representation, demonstrating that no changes were needed in the front end languages or DSLs. We adopt same general design approach here in Sections 4 and 5, in this case transforming the stencil dialect to a form that suits the Cerebras WSE.

4 Intermediate MLIR dialects for the WSE

There are two major groups of components comprising our MLIR based compiler flow for the WSE, the dialects and transformations that operate upon them. In this section we

propose three new MLIR dialects for the Cerebras WSE, `cs1-stencil`, `cs1-wrapper`, and `cs1-ir`. Figure 2 provides an overview of how these integrate, where the existing stencil dialect is lowered to the `cs1-stencil` dialect, which is then wrapped in the `cs1-wrapper` dialect. Lastly, this is all transformed into operations in the `cs1-ir` dialect.

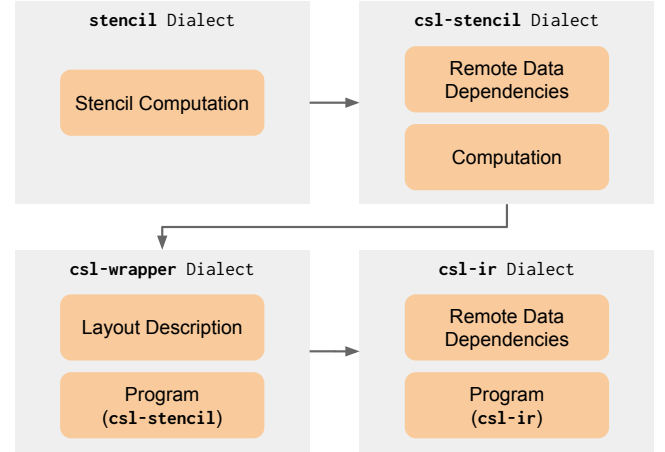


Figure 2. Through the use of three WSE MLIR dialects, we incrementally move from the stencil mathematical representation to the actor-based WSE execution model.

4.1 The `cs1-stencil` dialect

Stencils are lowered from the architecture-agnostic stencil dialect into the WSE-specific `cs1-stencil` dialect, which explicitly specifies the communications and stencil computation operations. The `cs1-stencil` dialect contains the following operations:

1. `cs1-stencil.prefetch` which describes the fetching of a single piece of remote data from its source into a local buffer.
2. `cs1-stencil.apply` which contains two regions, the first describes the processing of *chunks* of data received from remote dependencies, and the second region describes compute operations on data held locally, which is done with respect to any pre-processing already applied by the first region.
3. `cs1-stencil.access` based upon an offset from the current index will access a value from the stencil, either directly if this is held locally or from an internal buffer if held remotely. As such, this op may eventually be lowered to either a local memory access or to an access to a buffer into which incoming remote data is received.

Stencil computations by definition describe a reduction operation over a set of neighboring values (c.f. Listing 1). The `cs1-stencil` dialect enables us to perform a two-fold partial reduction where applicable. First, by supporting the reduction of remote data separately from data held locally, via the

`csl-stencil.apply` operation's first region. Secondly, by splitting communication into n chunks of data and reducing remote data on a chunk-by-chunk basis. In this case the first region is executed n times, but is able to immediately reduce incoming chunks of data into a single accumulator buffer, significantly lowering the overall memory requirements, of which each PE only provides a limited 48 kB. Supporting an optimization proposed by [20], which was shown to be highly efficient on the WSE, the `csl-stencil.apply` operation also enables constants to be applied to incoming data, rather than these being located in the compute region. Subsequently in the pipeline these are mapped to CSL primitives that apply constants at zero overhead to incoming data, effectively interleaving communication and compute.

4.2 The csl-wrapper dialect

CSL programs involve two main components, the program itself as well as a layout file which is a metaprogram that determines placement and routing of programs across the WSE, along with compile-time constants. CSL adopts a staged compilation approach where the compiler executes this layout file and will then use this to specialize and compile programs for individual cores. At this stage, any code labeled `comptime` in the PE programs is evaluated, parameters are lowered to constants, and the resulting PE program is aggressively optimized by the CSL SDK. If parameter values of PE programs diverge, all encountered variants are compiled separately. We provide the `csl-wrapper` dialect to handle this staged compilation approach, which is a domain-agnostic dialect that packages program-wide parameters, the layout metaprogram, and the PE program(s) together, alongside utility functionality to instantiate the program and metaprogram modules. It contains the following main operations:

1. `csl-wrapper.module` defines a module with optional program parameters and two regions. The first region controls *layout* across the WSE, and the second is for the program(s) to be mapped to PEs.
2. `csl-wrapper.import` will import CSL-specific libraries and helper functions, such as the CSL *memcpy* library which provides support for data transfers to and from the host.

Layout files, which are written in CSL, tend to comprise a loop nest iterating over the x and y dimensions of the WSE and individually placing kernels onto the PEs. Given that we run the same program on each PE, the layout in our approach corresponds to a view of this inner loop nest.

4.3 The csl-ir dialect

The `csl-ir` dialect is a direct re-implementation of a large subset of the CSL programming language, where constructs present in the language are represented in the same manner in `csl-ir`. This is because, ultimately, we invoke a printer

that generates CSL code from `csl-ir` intermediate representation. As such, all operations described above are eventually lowered into the `csl-ir` dialect and then from this CSL code is generated which is input to the Cerebras SDK compiler.

5 Compiler transformations for the WSE

The previous section presented both domain-specific and domain-agnostic intermediate dialects used to represent the structure of the IR at different stages during the transformation. Figure 3 sketches a detailed overall flow of the compilation pipeline with the main transformations that operate upon the IR where, starting from the Python or Fortran code provided as input to the Devito, PSyclone or Flang compilers, existing work [4, 9] is leveraged to transform these into the stencil dialect within the xDSL framework.

This flow involves executing a pipeline of transformations on the IR, modifying and refining it from one step to the next. We found that compiler flows for a machine such as the WSE challenges one of the underlying assumptions of MLIR. Work such as [4], targeting, conventional architectures is able to leverage MLIR's rich GPUs and CPUs based infrastructure, and it is highly effective to entirely lower out of the representation of the problem, in this case stencils, at an early stage in the pipeline. By contrast, we found that for dataflow, cache-less, architectures such as the WSE it was important to retain high level information about the nature of the kernel, in this case stencils, throughout the majority of the pipeline in order to be able to use this to most effectively generate a representation that most suits the WSE.

This section presents a strategy by which a stencil IR representation is iteratively lowered by applying our set of transformations, leveraging both domain-specific and domain-agnostic intermediate dialects described in Section 4. Listing 2 sketches the stencil MLIR representation, generated from the Fortran code in Listing 1 by [9]. At the initial stage of our compilation pipeline, the stencil dialect provides a mathematical description of the computation using value semantics, where variables represent independent copies of data without side effects or consideration of operational concerns such as how these are allocated to memory. A major contribution of our work is how this stencil representation is transformed by the compiler into a form that is optimized for the Cerebras WSE, and we use Listing 2 as a running example throughout this section to illustrate these transformations.

Listing 2 Example stencil code used as a running example in this section.

```
%result = "stencil.apply"(%input) ({
  ^0(%data : !stencil.temp<[-1,255]x[-1,255]x[-1,511]xf32>):
    %c0 = arith.constant 0.12345 : f32
    %d0 = stencil.access %data[1,0,0] : f32
    %d1 = stencil.access %data[0,0,0] : f32
    %t0 = arith.addf %d0, %d1 : f32
    %r0 = arith.mulf %c0, %t0 : f32
    "stencil.return"(%r0) : (f32) -> ()
}) : ... -> !stencil.temp<[0,254]x[0,254]x[0,510]xf32>
```

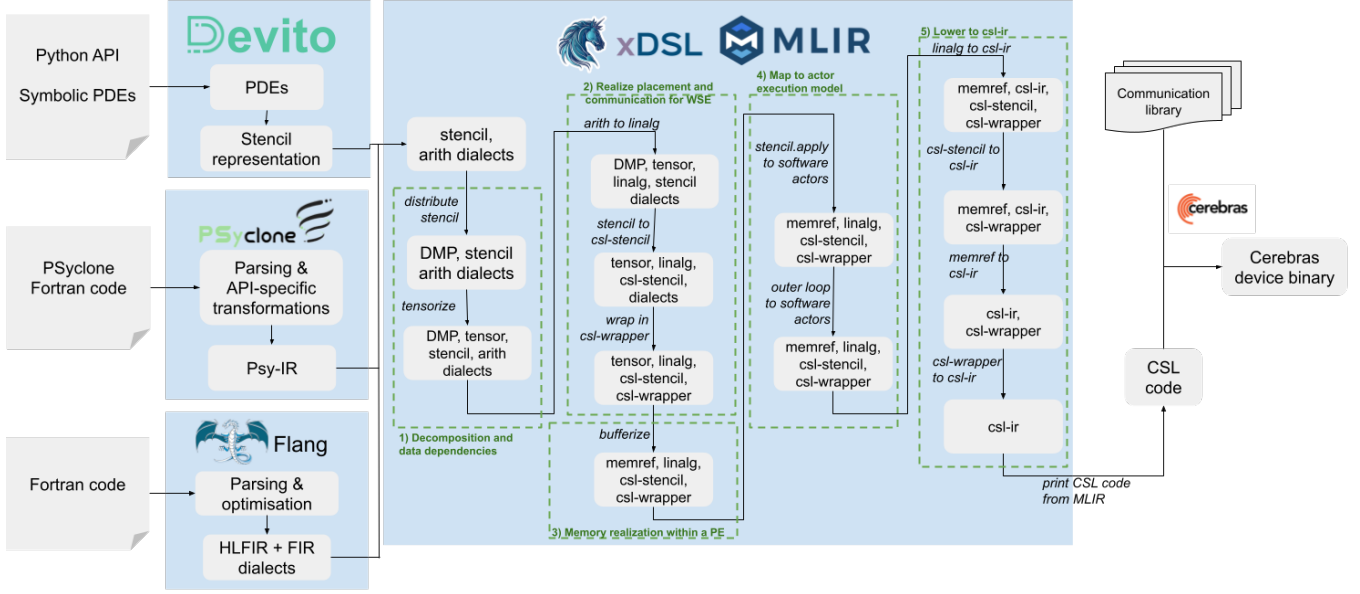


Figure 3. Our staged compilation flow extends existing infrastructure to target novel hardware through multiple stages, allowing efficient code generation for the Cerebras WSE.

As per the green dashed lines in Figure 3, there are five main groups of transformations that operate upon the IR to ultimately result in *csl-ir*, which can be compiled for the WSE. We now explore each of these groups.

5.1 Group 1: Decomposition and data dependencies

The first group of transformations in our approach decomposes the three-dimensional representation of the stencil onto the WSE’s two-dimensional grid of PEs and determines the data dependencies for stencil calculations on each PE. The overarching strategy is for a three-dimensional stencil of $x \times y \times z$ to be transformed into a two-dimensional stencil of size $x \times y$ such that each PE holds a column of z values. This choice of decomposition has been demonstrated to be effective on the WSE [20], and although mapping several columns of z values to each PE is theoretically possible, this approach was adopted due to the large number of PEs and small local memory size (48 kB).

Decomposition at the IR level is driven by the existing pass *distribute stencil* and we found that whilst this was initially intended for mapping stencils onto an HPC cluster via MPI [4], the same abstract logic also applies to decomposition across the WSE’s x and y sized grid. Ultimately, this results in the addition of the DMP dialect into our IR with `dmp.swap` operations inserted into the IR before each `stencil.apply` as shown in Listing 3. This operation indicates the necessary data exchanges that must occur before a specific apply can be performed, and as such describes remote data dependencies. The result of this transformation is a decomposition of the mathematical representation across the WSE, with

high-level communication operations denoting when data must be exchanged between PEs before a stencil operation can be performed.

Listing 3 After distributing x, y dimensions across PEs and tensorizing z dimension.

```
%1 = "dmp.swap"(%input) {strategy = #dmp.grid_slice_2d<
#dmp.topo<254x254>, false>, swaps = [...]}
(!stencil.temp<[-1,2]x[-1,2]xtensor<512xf32>>) ->
!stencil.temp<[-1,2]x[-1,2]xtensor<512xf32>>
%result = "stencil.apply"(%1) ({
^0(%d : !stencil.temp<[-1,1]x[-1,1]xtensor<512xf32>>):
%c0 = arith.constant <dense 0.123450> : tensor<510xf32>
%d0 = stencil.access %data[1,0] : tensor<510xf32>
%d1 = stencil.access %data[0,0] : tensor<510xf32>
%t0 = arith.addf %d0, %d1 : tensor<510xf32>
%r0 = arith.mulf %c0, %t0 : tensor<510xf32>
"stencil.return"(%r0) : (tensor<510xf32>) -> ()
}) : ... -> !stencil.temp<[0,1]x[0,1]xtensor<512xf32>>
```

The second transformation in this first group is to *tensorize* over the z -dimension, transforming the three-dimensional grid of FP32 scalars into a two-dimensional grid of FP32 tensors. The MLIR tensor dialect provides a way of representing abstract collections of data. This transformation maintains value semantics of the data representation, but also provides rank-polymorphism because operations in the arith dialect work across tensors of values, rather than scalars. This rewrite is illustrated in Listing 3, where converting the stencil into a two-dimensional stencil operating over tensors means that each stencil element (a column of tensors) is mapped to an individual PE on the WSE.

5.2 Group 2: Realize placement on and communication for WSE

This group of transformations maps the decomposed mathematical representation of the problem, that results from the first group of transformations, onto a physical representation of the WSE and converts data dependencies into communications between PEs. As described in Section 2, due to limited buffer sizes communications must occur in chunks and this also provides an opportunity to overlap communication and computation for performance. Consequently, the *stencil* to *cs1-stencil* transformation converts from the *stencil* to the *cs1-stencil* dialect which involves all `dmp.swap` operations being replaced by *cs1-stencil.prefetch*. These *cs1-stencil.prefetch* operations are added into their corresponding *cs1-stencil.apply* which itself combines both compute and communication.

The compute region of *stencil.apply* is split into two, the first region describing compute based on chunks of data that are held remotely and the second region on data held locally. This is illustrated in Listing 4, where communication involves two chunks, with the resulting *acc* accumulator from the first region then consumed by the second region. There are two purposes to the first, remote data handling, region firstly the packing data, in the case of Listing 4 inserting the received chunk of data into the result tensor, and secondly undertaking optional compute on received data before it is packed.

Listing 4 After transformation from the *stencil* to *cs1-stencil* dialect.

```
%result = "cs1_stencil.apply"(%input) <{swaps =
  [#cs1_stencil.exchange<to [1, 0]>], num_chunks = 2 : i64> <{
    ^0(%d : tensor<4x255xf32>, %offset : index, %acc : tensor<510xf32>):
      %0 = cs1_stencil.access %d[1, 0] : tensor<255xf32>
      ...
      %1 = tensor.insert_slice %0 into %acc[%offset][255][1]
      cs1_stencil.yield %1 : tensor<510xf32>
  }, {
    ^1(%d : !stencil.temp<[-1,1]x[-1,1]xtensor<510xf32>,>
      %acc : tensor<510xf32>):
      %c0 = arith.constant <dense 1.234500e-01> : tensor<510xf32>
      %d0 = cs1_stencil.access %d[0,0] : tensor<510xf32>
      %t0 = arith.addf %acc, %d0 : tensor<510xf32>
      %r0 = arith.mulf %t0, %c0 : tensor<510xf32>
      cs1_stencil.yield %r0 : tensor<510xf32>
  }) : ...
```

The other transformation in this group, *wrap cs1-stencil* in *cs1-wrapper*, generates the layout metaprogram which maps kernels to PEs on the WSE, along with program-wide constant parameters extracted from the *cs1-stencil* operations. Although *cs1-wrapper* is a domain-agnostic dialect, this pass nevertheless populates it with highly domain-specific information from the *cs1-stencil* dialect, which it wraps and retains.

5.3 Group 3: Memory realization within a PE

Until this point compute operations on the stencil have been expressed as value semantics where the focus is on the values themselves and operational details, such as where these are held in memory, is abstracted. This group of transformations converts the IR into reference semantics, which involves mapping tensors to physical memory allocation by way of transforming them to *memrefs*. A major reason for doing this is that CSL mathematical operations follow the Destination-Passing Style (DPS) form, where they operate on physical memory, reading inputs from and storing results to physical memory that has been passed as operands. By contrast, tensors are immutable, where arithmetic operations involving tensors always result in a new tensor.

A transformation pass rewrites our proposed dialects to reference semantics, undertaking *partial bufferization*, before MLIR's bufferization is used to convert tensors to *memrefs*. If a computation requires additional intermediate buffers then MLIR's bufferization pass acts as a fail-safe that will automatically allocate these as required. Indeed, this demonstrates one of the benefits of using MLIR more generally, where extensive functionality is already provided by the MLIR ecosystem for common operations such as materialization, and this can be reused rather than having to reinvent it within an entirely bespoke compiler stack.

Listing 5 After transformations in the third group of dialects that realize memory within a PE.

```
%result = "cs1_stencil.apply"(%input) ... ({ ... }, {
  ^1(%d : !stencil.temp<[-1,1]x[-1,1]xmemref<510xf32>,>
    %acc : memref<510xf32>):
    %c0 = arith.constant <dense 1.234500e-01> : memref<510xf32>
    %d0 = cs1_stencil.access %d[0,0] : memref<510xf32>
    linalg.add ins(%acc, %d0) outs(%acc)
    linalg.mul ins(%acc, %c0) outs(%acc)
    cs1_stencil.yield %acc : memref<510xf32>
  }) : ...
```

The *arith* dialect does not support CSL's DPS form of operations, whereas the linear algebra *linalg* dialect does. We therefore convert compute operations in the *arith* dialect to their counterparts in the *linalg* dialect. Listing 5 illustrates the IR after these transformations have been run, where it can be seen that *memrefs* replace tensors and these are provided to *linalg* operations as operands. Furthermore, to make best use of the limited amount of memory on the WSE, the *acc* *memref* is reused in Listing 5 as input, to hold intermediate results, and final calculation results.

5.4 Group 4: Map to the actor execution model

Figure 1 illustrated that in order to communicate within a CSL program one must structure their code as tasks that are called when corresponding messages arrive. It was highlighted in Section 2.1 that it is effective to view the WSE

as following the actor model, where PEs are hardware actors and tasks software actors. This group of transformations lowers to the WSE’s asynchronous execution model by splitting up `csl-stencil.apply` operations into their constituent activities and mapping these to actors. Because `csl-stencil.apply` operations perform asynchronous communication on the WSE, their remote-data and local-data subregions are lowered into software actors which are activated, respectively, each time chunks of remote data are received, and once at the end when all chunks from all neighbors have been received. This is very similar to the `stencil.communicate` function called by `seq_kernel0` and `seq_kernel1` in Figure 1.

The second actor resulting from this lowering to actors typically processes data held locally and is executed exactly once as the last part of the operation. Similarly to Figure 1, it is from this second task that control flow proceeds, where the successive function is invoked by this software actor.

As described in Section 2, to enable naturally synchronous stencils to operate in an asynchronous fashion one is unable to have a top-level timestep loop, or even successive `csl-stencil.apply` operations without a loop. This is because the `csl-stencil.apply` operation triggers asynchronous communications after which program flow does not continue in a synchronous manner, and both these configurations are required by the benchmarks explored in Section 6. The second transformation of this group converts the top-level control flow enclosing `csl-stencil.apply` operations into a control-flow task graph, comprising software actors which consist of callable zero-parameter functions as well as local tasks in our `csl-ir` dialect. For instance, a top-level timestep `scf.for` loop is lowered into a control-flow graph consisting of CSL functions instead of basic blocks.

5.5 Group 5: Lowering to the `csl-ir` dialect

The last group of transformations will lower to the `csl-ir` dialect from which CSL code is printed out. The first transformation of this fifth group, `linalg to csl-ir`, lowers compute operations, which are in the `linalg` dialect, into corresponding operations in the `csl-ir` dialect. These operations are expressed across arrays of data, and the naive approach would be to generate a loop that performs an arithmetic operation for each element. However, best practice on the WSE is to instead use CSL’s Data Structure Descriptors (DSDs), which are essentially affine iterators over buffers via native hardware support. CSL provides a set of high-throughput DSD arithmetic builtin functions that deliver a significant performance improvement compared to computations that are not based on DSD builtins. For instance, via DSDs, the multiply and add operations shown in Listing 5 are lowered to `@fmacs(dest, src1, src2, f32_val)`, providing an FP32 fused multiply-accumulate.

At this point, the stencils and compute components have been transformed into the `csl-ir` dialect. However, memory is still represented by the `memref` dialect. While `memref`

types are used throughout the `csl-ir` dialect, operations on `memrefs`, such as allocation and deallocation, need to be lowered to the `csl-ir` dialect. Furthermore, DSDs are initialized as views and subviews onto buffers. Consequently, the `lower memref to dsd` pass generates DSD definitions on top of `memref` buffer allocations that are used throughout builtin compute functions as well as for communication.

The last step of this group lowers the top level `csl-wrapper.module` operation to `csl-ir`. This transformation generates two output modules, the first comprises `csl-ir` operations for the layout metaprogram from its abstract representation, and the second is the `csl-ir` operations generated in preceding passes.

5.6 Runtime communications library

Data exchanges between hardware actors (PEs) are handled by a CSL-based library that we designed using the partitionable communication strategy developed by Jacquelin et al. [20] for star-shaped stencils of up to three dimensions at variable stencil sizes. The library encapsulates boiler-plate code for sending and receiving data split into chunks of configurable sizes. Initially, it schedules asynchronous send and receive data exchanges in all four directions, using multiple internal *tasks* per direction to handle the completion of asynchronous communication steps and updating of routing patterns, and which in turn trigger user-provided callbacks. CSL’s staged compilation mechanism provides sufficient flexibility and power to compile arbitrary star-shaped stencil patterns, allowing us to focus on code generation on the software-actor level, internal to each PE. Supporting different stencil shapes (for instance, box-shaped communication patterns) would require a library update. Alternatively, in work complementary to ours Sai et al. [40] present software-based routing patterns, whereas code generation for hardware routing configurations and switching updates remains future work, even in the confined case of stencil patterns.

5.7 Optimization passes

Whilst much of this section has focused on transformations to recast the mathematical stencil representation to a form that suits execution on the WSE, several additional transformations were introduced solely for the purpose of optimization at the stencil and arithmetic levels. The first of these, `stencil-inlining`, is executed at the start of the compilation pipeline and attempts to merge consecutive `stencil.apply`s into a single fused kernel. This is an existing pass that was developed in [4] to remove the unnecessary overhead of host-device context switching between stencils that are consecutive. The transformation ensures all outputs of the first stencil are passed through to the second, even if values are unaffected, and the operations performed by both stencils are then merged into a single `stencil.apply`. Being able to leverage this existing transformation is an example of

one of the key benefits of integrating with a wider compiler ecosystem such as MLIR, as it avoids duplication of effort developing common passes. In the case of the UVKBE benchmark, that we explore in Section 6, this pass is able to merge all `stencil.apply`s into one single operation. The fused kernel is subsequently split later in the pipeline according to buffer communications, and to allow for potential interleaving of communication and computation.

`Varith` is an xDSL-specific dialect which represents multiple arithmetic additions or multiplications as a single variadic arithmetic operation. We employ this dialect early in the compilation pipeline to represent arithmetic operations where possible because it significantly reduces the complexity associated with manipulating them, e.g. when splitting the computation into regions for separate processing of locally and remotely held data. Furthermore, the `varith` dialect is required by the `varith-fuse-repeated-operands` transformation which we leverage to convert a series of consecutive additions of the same argument into a multiplication. This optimization is especially important for codes such as the Acoustic kernel that we explore in Section 6, as it can eliminate multiple arithmetic operations, in the case of that kernel we were able to replace three DSD addition operations with a single multiplication.

Moreover, the `varith-fuse-repeated-operands` transformation also enables us to fuse multiplications and additions into an `fmac` operation. When this transformation is used in conjunction with `linalg` dialect to model DSD operations, it makes it possible for our `linalg-fuse-multiply-add` transformation to identify multiplication and addition operations that can be combined. Ultimately, the result of applying this transformation is that the `@fmacs` CSL intrinsic, that was mentioned in Section 5.3, is generated for suitable chains of arithmetic operations. Due to the prevalence of a multiplication followed by addition in stencils, we have found that this optimization results in a large number of multiplication and addition operations being converted into `@fmacs` which delivers improved performance.

An efficient lower-level optimization strategy is to check whether the same reduction function is applied across the entire stencil shape. In this case, we can generate highly optimized code using tensor broadcasting techniques on the accumulator buffer, expanding it with virtual dimensions that match the communication buffer, so as to perform a one-shot reduction of the entire communication buffer in a single builtin function call. Conversely, if points in the stencil shape needs to be processed using different functions, or if a subset of points is altogether ignored, the compiler instead needs to obtain separate pointers into the communication buffer and generate individual builtin function calls to process communicated data, which is less performant. Another low-level optimization is to promote coefficient application into the communication code where possible, to interleave

communication and computation. Effectively, instead of moving incoming data from an input queue to PE memory with a `@mov`-like builtin, we can instead multiply incoming data from the input queue to memory, by a set of coefficients. These two low-level techniques can be used in combination.

6 Evaluation

In this section we compare the performance of our approach on both a Cerebras CS-2 and CS-3 system against CPU and GPU baselines. For each benchmark, we considered three different problem sizes, classified by the number of elements in the X and Y dimensions as: small – 100×100 , medium – 500×500 and large – 750×994 . The large size was chosen to fully occupy the PE grid on the WSE2. The Z dimension varies for each of the five following benchmarks:

Jacobian (*Flang*) is extracted from Fortran code solving Laplace’s equation for diffusion in 3D. This is a six-point stencil, and is run for 100,000 iterations with 900 elements in the Z dimension.

Diffusion (*Devito*) solves the heat diffusion equation. Expressed in Devito’s Python-based DSL, this is a 3D 13-point stencil running for 512 iterations with 704 elements in the Z dimension.

Acoustic (*Devito*) models the isotropic acoustic wave equation in Devito using a 2nd order approximation in time. Similar to ‘Diffusion’ this is a 3D 13-point stencil, run for 512 iterations with 604 elements in the Z dimension.

25-Point Seismic (*Cerebras*) is a seismic kernel written in CSL using a 1st order approximation in time. Translated from [Jacquelin et al.](#), this kernel provides a direct comparison against manually tuned CSL. This is a 3D 25-point stencil, ran for 100,000 iterations with 450 elements in the Z dimension.

UVKBE (*PSyclone*) includes four fields, two of which need to be communicated across PEs as well as two consecutive `stencil.apply` operations. This benchmark is written in Fortran via the PSyclone DSL and is run for a single iteration with 600 elements in the Z dimension.

For all benchmarks, we use 32-bit single-precision floating point. Reported results are the average throughput measured in GPs/s (a.k.a GCells/s) over three runs. It should be highlighted that Figures 4, 5 and 6 include error bars (CI 95%) but these are very small because there were no significant outliers between runs. We use xDSL version v0.35 and target Cerebras SDK 1.3.0 and Cerebras ML Software 2.4. Figure 4 reports performance across four of these benchmarks with the large problem size from Flang (Jacobian), Devito (Diffusion), Cerebras (Seismic), and PSyclone (UVKBE) using our compilation approach on the WSE2 and WSE3. Benchmarks are run for 100000, 512, 100000 and 1 iteration(s) respectively. The major reason for the performance difference between

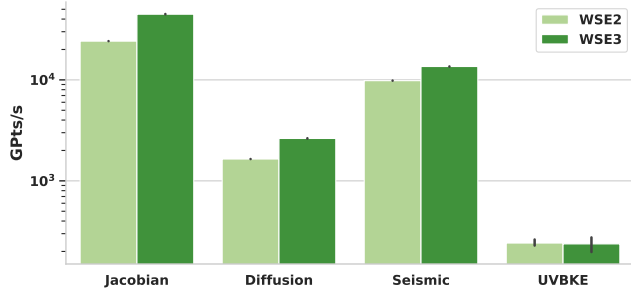


Figure 4. Performance comparison of WSE2 and WSE3 across benchmarks running the large problem size using our approach. Our code generator exploits the latest hardware features for optimal performance.

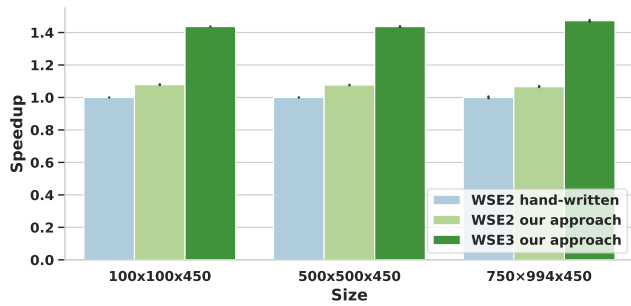


Figure 5. Our compiler-based approach for the seismic benchmark plotted against the hand-written equivalent developed by Cerebras [19]. Efficient communication and task utilization gives our generated code the performance edge.

the WSE2 and WSE3 is the utilization of the WSE3’s upgraded switching logic. Hardware limitations on the WSE2 in the switch configuration require each PE to transmit data to itself as well as to its neighbors (on each of the east, west, north, and south routes) [20]. On the WSE3 this is no longer necessary, and our WSE3 communications library exploits this architectural improvement.

6.1 Our approach vs hand-written WSE code

In collaboration with Cerebras engineers, Jacquelin et al. [20] developed a hand-written, optimized seismic modeling benchmark based on a 25-point stencil. While available in the Cerebras SDK for the WSE2, it is not supported on the WSE3 [19]. Figure 5 reports performance of this hand-written kernel on the WSE2 for different problem sizes in comparison to an equivalent implementation generated by our approach for the WSE2 and WSE3. [20] reported that the hand-written implementation achieved 28.2% of theoretical peak performance on the WSE2, and the version generated by our compilation pipeline outperforms this by up to 7.9%. The performance is affected by several factors. Our generated version is more memory efficient, allowing communication in a single chunk

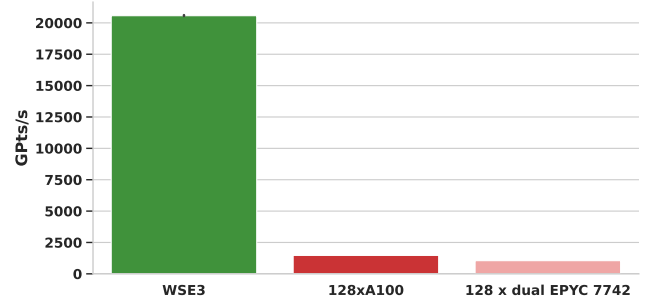


Figure 6. Performance of Devito acoustic benchmark on the WSE3 (large problem size, 100,000 iterations) compared to MPI + OpenACC kernels on 128 A100 GPUs on Tursa and MPI + OpenMP kernels on 128 CPU nodes on the ARCHER2 Cray-EX, from [5]. A single wafer delivers throughput as a large GPU or CPU cluster, showcasing high throughput capabilities of the WSE3.

where the hand-written version uses two. However, data from Jacquelin et al. suggests that this may not account for all performance differences. Our compilation approach only communicates data that is required by the calculation. For this benchmark, this means that the first and last column values are not communicated, whereas the hand-written code transmits the full column of values, including those which are not required. Our communications library also manages tasks more efficiently, reducing the total number of tasks required by approximately 50%, which reduces task activation overhead.

The hand-written code is restricted to the WSE2 due to architectural hardware generations that require kernel rewriting. By contrast, our approach seamlessly supports both the WSE2 and WSE3. Overall, the code generated for the WSE3 outperforms the WSE2 implementation by up to 38.1%, primarily due to the architectural improvements described previously. As described in Section 7, [40] proposed a Python-based stencil DSL for the WSE leveraging an entirely bespoke compilation pipeline. Their compiler is not publicly available, but in [40] they compare against the same manually implemented 25-point stencil kernel as Figure 5. They report marginally worse performance than the manually implemented code, and the argument made in [40] is around programmer productivity. By contrast, as demonstrated in this section, for this same kernel our approach delivers both programmer productivity and performance advantages.

6.2 Performance on WSE3 against GPUs and CPUs

To provide a comparison of our approach on the WSE3 against traditional HPC architectures, GPUs and CPUs, we evaluated the performance of Devito’s *acoustic* benchmark against results reported by [5]. Their study evaluated MPI + OpenACC kernels on 128 Nvidia A100 GPUs on the Tursa

supercomputer, and MPI + OpenMP kernels on 128 CPU nodes of the ARCHER2 CPU based Cray-EX supercomputer. Each Tursa node contains two 24-core, AMD 7413 EPYC processors and four NVIDIA Ampere A100-80 GPUs connected via NVLink, and four 200 Gbps Infiniband links connecting the nodes. Each ARCHER2 node contains two 64-core, AMD Zen2 (Rome) EPYC 7742 processors, nodes, connected via HPE's proprietary Slingshot interconnect. Experimental setup used version 23.5-0 of Nvidia's nvc++ compiler on Tursa, and Cray's Clang version 11.0.4 on ARCHER2. All experiments used single-precision floating-point arithmetic.

Bisbas et al. used significantly larger grid sizes, 1024^3 and 1158^3 points on the CPU and GPU respectively, giving them a slight advantage by lowering communication overhead. Figure 6 demonstrates that the WSE3 achieves a significantly faster throughput for a specific problem size, leading the race for a faster solution against 128 A100s and 128 CPU nodes (16384 CPU cores). The WSE3 is around 14 times faster to solution than 128 A100s, and 20 times against 128 CPU nodes.

We acknowledge that this is not a strictly direct comparison due to the differing problem sizes. In the strong scaling study by [5], decomposing the problem across GPUs results in each device computing a smaller sub-domain with a higher communication-to-computation ratio, thereby preventing full exploitation of single-GPU potential. By contrast, the WSE3 processes the entire domain on a single wafer. While a weak-scaling comparison—where each GPU solves a larger problem—would likely reduce the WSE3's speedup, the advantage would remain significant. Furthermore, the GPU baseline relied on OpenACC rather than hand-tuned CUDA (available only in proprietary versions of Devito), thus, not exercising its full potential. Nevertheless, this experiment is yet another demonstration that in the race for "time-to-solution for a specific problem size", the WSE3 outperforms large clusters of traditional hardware [33, 41, 44].

Indeed, if one were to assume perfect scaling on the CPU then around 50% of the ARCHER2 supercomputer would be required to match performance of a single WSE3 for this benchmark. Furthermore, whilst these performance numbers are impressive, it is also important to highlight that the Devito code here is running unchanged across the WSE3, GPUs and CPUs.

6.3 WSE3 roofline performance

A roofline plot visualizes a kernel's performance in FLOPs/s against its arithmetic intensity in FLOPs/byte, bounded by the system's peak compute and memory bandwidth limits. The dashed lines represent the theoretical maximum of the machine, and applications that fall under the initial diagonal part of this line are bound by memory performance, and those under the horizontal part are limited by compute. The higher the performance of a benchmark then the closer it is to the theoretical maximum. Indeed, it was a roofline plot

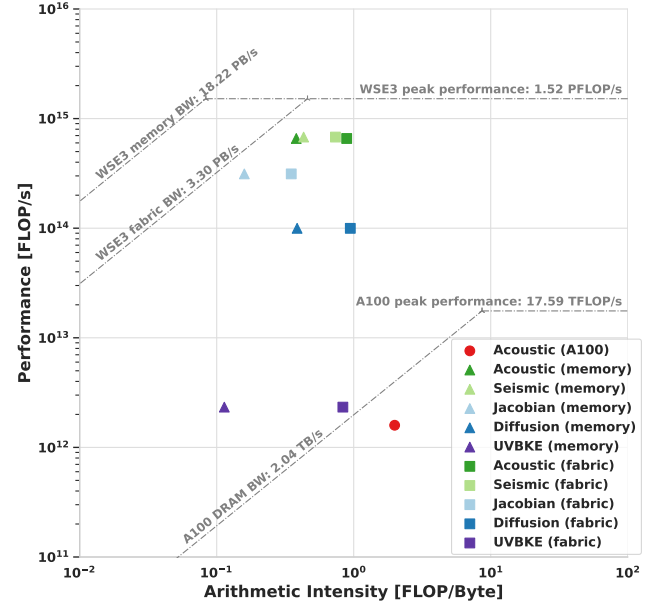


Figure 7. Roofline plot of our five benchmarks on the WSE3 and the acoustic benchmark on a single A100.

that Ltaief et al. [28] used to argue the memory benefits of the WSE, as they observed codes moved from under the diagonal portion, memory bound, on other architectures to the horizontal part, compute bound, on the WSE.

Figure 7 plots the roofline of the WSE3 with our five benchmarks and the acoustic benchmark on a single A100 GPU. On the WSE3 local memory has a higher bandwidth than the communication fabric, and so for each WSE benchmark we plot two values; the first based on all data accesses being from local memory and the second all data accesses being via fabric. It can be seen that for all WSE3 benchmarks that assume data resides in memory then the benchmarks are compute bound, and all apart from the Jacobian are also compute bound when assuming all data accesses are via the fabric. By contrast, we also plot the acoustic benchmark on the A100 GPU from [5] and it can be seen that this code is memory bound on the A100. Effectively, the approach described in this paper has enabled the user to move their kernel to an architecture which delivers much better performance, a major reason being that the code is no longer memory bound, without requiring any application-level changes. Jacquelin et al. [20] observed compute-bound performance for a 25-point seismic stencil, and our experiments show that the same finding applies to a larger variety of stencil programs.

6.4 Developer productivity

As described in Section 2, writing code for the WSE is difficult and adds significant complexity not least because the

programmer must rethink their execution model. Consequently, from a programmer productivity perspective, it can take significant time to develop and optimize code for the architecture [8]. However whilst it does not provide a full picture, a lines of code, LoC, metric provides some indication as to the effort required.

Benchmark	CSL kernel only (LoC)	CSL entire (LoC)	DSL & our approach (LoC)
25-point Seismic	196	980	81
Acoustic	211	995	81
Diffusion	192	976	40
Jacobian	180	964	28
UVKBE	203	987	44

Table 1. Lines of Code (LoC) comparison between generated CSL and programming using a DSL with our compilation approach.

Table 1 reports a lines of code comparison for each benchmark between the generated CSL and that required when writing code in a DSL and using our compilation approach. We provide two numbers for the CSL code, *CSL kernel only* comprises the kernel alone without any support for placement on the WSE, communications or interaction with the host. *CSL entire* is the full CSL code length containing both the kernel and all the required support functionality in our runtime library that was described in Section 5.6. It can be seen that, regardless of the benchmark and DSL, using our approach requires significantly reduces the amount of code that must be written. Furthermore, it should be highlighted that the DSL source code would likely be much more accessible for a scientific programmer to understand, whereas not only is CSL very architecture specific but it also tends to be difficult to differentiate the scientific stencil code from all the WSE specific implementation details that surround it.

7 Related Work

Rocki et al. [36] explored stencil codes on the first iteration of the WSE, the CS-1. The authors ported the main components of the iterative BiCGSTAB Krylov subspace method, the SpMV, AXPY and inner product, to the WSE. This work was before Cerebras released CSL and so was undertaken using a proprietary programming interface, achieving 0.86 PFLOPS, over 200 times the performance of the NETL Joule supercomputer. Recognizing the programmability challenge of the WSE, Woo et al. [47] followed up this early work by developing a Python-based API for programming the WSE. Crucially, their API maintained the performance of the CS-2 by delivering over two orders of magnitude runtime reduction on the WSE2 compared to the NETL Joule 2.0 supercomputer. Our work addresses two major limitations

of Woo et al. [47], firstly in their approach programmers must convert their code to use a specific API, whereas our work places the emphasis on the DSL developer to integrate with the stencil dialect and then, without any user code changes, these will run on the WSE. The second major difference is that Woo et al. [47] generated assembly level code, which itself is under NDA, rather than CSL. Instead, by targeting CSL, our approach is portable between generations, such as WSE2 to WSE3, and we can also publish the source code of the compiler, as it relies only on public APIs.

A Python-based stencil DSL for the WSE was proposed by Sai et al. [40] which, similarly to our approach, produces CSL code. Their focus is largely on generating a software-based routing algorithm for different stencil shapes in a manner complementary to our work, whereas the generation of compute or overarching program structure is not discussed. Similarly to Woo et al. [47], they developed their own entirely bespoke compiler whereas our approach provides a series of building blocks within the MLIR ecosystem. Furthermore, unlike previous work (see [40, 47]) we decouple the frontend programming technology from the compilation flow targeting the WSE, therefore application code can remain unchanged rather than first needing to port into a new DSL.

SPADA [14] is a spatial dataflow language targeted at the WSE, specifically the CS-2, where Gianinazzi et al. developed a bespoke compiler and integrated the GT4Py stencil DSL atop this. SPADA and our approach are complementary, where SPADA heavily optimizes the lower levels of the stack by providing a WSE specific abstraction and optimizations. Conversely, by leveraging MLIR we progressively lower through multiple abstraction levels towards CSL, exploiting each of these levels and existing compiler building blocks where applicable to generate the most appropriate target code for both CS-2 and CS-3 versions of the WSE architecture. Fundamentally, SPADA aims to solve a different problem to our work, where we prioritize sharing of infrastructure between frontends to promote ease of integration. Consequently, there is a greater semantic gap in [14] between the GT4Py stencil DSL and SPADA, requiring more work in each of the frontends to connect stencils to SPADA and risking duplication. By contrast using our approach once the stencil dialect has been generated then our compilation pipeline handles the entirety of the transformation from this to the WSE in a frontend agnostic manner. Indeed, it would be interesting in future work to integrate our work with SPADA, if they were to develop an MLIR dialect, as this would enable stencil DSL frontends to transparently exploit the WSE specific abstractions and optimizations developed by Gianinazzi et al.

Whilst differences in experimental setup make performance comparisons somewhat difficult, given the complementary nature of SPADA and our work it is still interesting to highlight key aspects. Using our approach, as described in Section 6.3, a 3D Jacobian which implements LaPlace's

equation for diffusion delivers 169 TFLOP/s on the CS-2 and 313 TFLOP/s on CS-3, compared to a 2D Laplacian in [14] delivering 120 TFlop/s on the CS-2. UVKBE is the highest performing kernel in [14] delivering around 150 TFlop/s on the CS-2. This is greater performance than the UVKBE benchmark in our work, but we adopted the UVKBE benchmark from [15] which only executes one iteration. This is different from UVKBE in [14] which has more opportunity to ameliorate overhead. Conversely, our highest performing kernel (25-point stencil) delivers 491 TFlop/s on the CS-2 and 678 TFlop/s on the CS-3. Irrespective, Gianinazzi et al. also demonstrate the significant programmer productivity benefits of driving the WSE from a stencil based DSL and deliver impressive reductions in the lines of code required compared to manually written CSL.

Jacquelin et al. [20] developed an optimized implementation of a 25-point stencil for seismic modeling on the CS-2, and Sai et al. [39] implemented a 3D 9-point stencil, demonstrating how data can be indirectly transferred to PEs at diagonal offsets to support diagonal data dependencies for stencils of rank 1. Furthermore, Sai et al. [38] present a finite volume kernel that performs a three-dimensional 7-point stencil operation followed by a whole-fabric all-reduce operation. A cornerstone of our approach is to fold the techniques and best practices in these papers into the compiler, enabling non-expert programmers to benefit from such insights without requiring expertise in programming the WSE.

The scientific computing community has long explored abstraction layers to improve productivity, spanning high-level domain-specific approaches and low-loop/expression-level optimizations. Higher-level symbolic frameworks include the Python-based OpenSBLI [18] which employs Einstein notation for symbolic modeling, automating parallelism via the C/C++-based OPS library [35]. ExaStencils [21, 25] combines layered symbolic DSLs for finite differences, supporting CPU/GPU architectures through code generation. STELLA [16], and subsequently GridTools [2], uses mathematical discretization syntax notably enabling GPU acceleration for the COSMO weather model [43].

However, none of these existing stencil frameworks support architectures such as the WSE, and in this work we demonstrate one of the major benefits of MLIR, where a range of frontend languages and DSLs can benefit from building blocks which lower an existing stencil dialect to the WSE. Indeed, stencil DSLs such as those above could integrate with our approach. Furthermore, whilst our approach is the first MLIR pipeline for the Cerebras WSE, there are several MLIR dialects and transformations developed for other novel architectures such as AMD's AIEs [3], Tenstorrent Tensix [42], and Wang et al. [46] provide MLIR infrastructure for CGRA compilers. However, whilst these also provide MLIR building blocks for these different architectures, unlike our approach on the Cerebras WSE, none of them support

acceleration of unmodified code in existing languages and DSLs on the target accelerator.

8 Conclusions and further work

In this paper we have presented an MLIR lowering pipeline for stencil codes, demonstrating how one can effectively target the Cerebras WSE in code generation based upon existing high-level stencil domain specific abstractions without code level changes. We evaluate using five benchmarks from three different programming frontends, and demonstrated that our approach is competitive to, and can out-perform, a stencil hand-tuned, in part, by Cerebras engineers, and in comparison to a cluster of 128 Nvidia A100 GPUs and 128 nodes of the CPU-based ARCHER2 Cray-EX supercomputer managed to out-perform these by 14x and 20x, respectively.

Prior to this work considerable effort has always been required to port a code to WSE3, not just requiring a rewrite of the code in CSL but furthermore a fundamental recasting of the algorithm to suit the asynchronous, actor-based nature of the Cerebras system. During this work we made the observation that adopting the actor-based paradigm helps provide an execution model of communication and computation on the WSE that enables consistency. In future work, we consider this important to be explored further by the community, not just as the potential programming model for the WSE but moreover how it might characterize some of the problems encountered when targeting a range of other CGRA-like accelerators. Our implementation is available as open-source under <https://github.com/xdslproject/wse-stencil> which uses v0.35 of xDSL that contains many of the dialects and transformations discussed in this paper <https://github.com/xdslproject/xdsl>.

Acknowledgments

This work was supported by the Engineering and Physical Sciences Research Council (EPSRC) grant EP/W007940/1 and a Royal Society of Edinburgh personal research fellowship award number 3271. We thank the Edinburgh International Data Facility (EIDF) at the University of Edinburgh for access to the CS-3 and Leibniz Supercomputing Centre (LRZ) for the CS-2. The authors would like to thank Cerebras for providing technical support, along with P. Kelly, E. Bauer and N. Heinimann for their thoughtful early feedback, and finally the shepherd and reviewers for their comments and support improving this work.

References

- [1] Samantha V Adams, Rupert W Ford, M Hambley, JM Hobson, I Kavčič, Christopher M Maynard, Thomas Melvin, Eike Hermann Müller, S Mullerworth, Andrew R Porter, et al. 2019. LFRic: Meeting the challenges of scalability and performance portability in Weather and Climate models. *J. Parallel and Distrib. Comput.* 132 (2019), 383–396.
- [2] Anton Afanasyev, Mauro Bianco, Lukas Mosimann, Carlos Osuna, Felix Thaler, Hannes Vogt, Oliver Fuhrer, Joost VandeVondele, and

- Thomas C. Schulthess. 2021. GridTools: a framework for portable weather and climate applications. *SoftwareX* 15 (2021), 100707. doi:10.1016/j.softx.2021.100707
- [3] AMD. [n. d.]. *IRON API and MLIR-based AI Engine Toolchain*. <https://github.com/Xilinx/mlir-ae>
- [4] George Bisbas, Anton Lydike, Emilien Bauer, Nick Brown, Mathieu Fehr, Lawrence Mitchell, Gabriel Rodriguez-Canal, Maurice Jamieson, Paul H. J. Kelly, Michel Steuwer, and Tobias Grosser. 2024. A shared compilation stack for distributed-memory parallelism in stencil DSLs. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 38–56. doi:10.1145/3620666.3651344
- [5] George Bisbas, Rhodri Nelson, Mathias Louboutin, Fabio Luporini, Paul H. J. Kelly, and Gerard Gorman. 2025. Automated MPI-X Code Generation for Scalable Finite-Difference Solvers. In *2025 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 689–701. doi:10.1109/IPDPS64566.2025.00067
- [6] Nick Brown. 2021. Accelerating advection for atmospheric modelling on Xilinx and Intel FPGAs. In *2021 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 767–774.
- [7] Nick Brown. 2024. Fully integrating the Flang Fortran compiler with standard MLIR. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 939–949.
- [8] Nick Brown, Brandon Echols, Justs Zarins, and Tobias Grosser. 2022. Exploring the suitability of the cerebras wafer scale engine for stencil-based computation codes. In *European Conference on Parallel Processing*. Springer, 51–65.
- [9] Nick Brown, Maurice Jamieson, Anton Lydike, Emilien Bauer, and Tobias Grosser. 2023. Fortran performance optimisation and auto-parallelisation by leveraging MLIR-based domain specific abstractions in Flang. In *Proceedings of the SC '23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis* (Denver, CO, USA) (SC-W '23). Association for Computing Machinery, New York, NY, USA, 904–913. doi:10.1145/3624062.3624167
- [10] Denis Caromel, Ludovic Henrio, and Bernard Paul Serpette. 2009. Asynchronous sequential processes. *Information and Computation* 207, 4 (2009), 459–495.
- [11] Johannes de Fine Licht, Maciej Besta, Simon Meierhans, and Torsten Hoefler. 2020. Transformations of high-level synthesis codes for high-performance computing. *IEEE Transactions on Parallel and Distributed Systems* 32, 5 (2020), 1014–1029.
- [12] Jonathan Edwards. 2009. Coherent reaction. In *Proceedings of the 24th ACM SIGPLAN Conference Companion on Object Oriented Programming Systems Languages and Applications* (Orlando, Florida, USA) (OOPSLA '09). Association for Computing Machinery, New York, NY, USA, 925–932. doi:10.1145/1639950.1640058
- [13] Mathieu Fehr, Jeff Niu, River Riddle, Mehdi Amini, Zhendong Su, and Tobias Grosser. 2022. IRDL: an IR definition language for SSA compilers. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (San Diego, CA, USA) (PLDI 2022). Association for Computing Machinery, New York, NY, USA, 199–212. doi:10.1145/3519939.3523700
- [14] Lukas Gianinazzi, Tal Ben-Nun, and Torsten Hoefler. 2025. SPADA: A Spatial Dataflow Architecture Programming Language. *arXiv preprint arXiv:2511.09447* (2025).
- [15] Tobias Gysi, Christoph Müller, Oleksandr Zinenko, Stephan Herhut, Eddie Davis, Tobias Wicky, Oliver Fuhrer, Torsten Hoefler, and Tobias Grosser. 2021. Domain-specific multi-level IR rewriting for GPU: The Open Earth compiler for GPU-accelerated climate simulation. *ACM Transactions on Architecture and Code Optimization (TACO)* 18, 4 (2021), 1–23.
- [16] Tobias Gysi, Carlos Osuna, Oliver Fuhrer, Mauro Bianco, and Thomas C. Schulthess. 2015. STELLA: a domain-specific tool for structured grid methods in weather and climate models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2015, Austin, TX, USA, November 15-20, 2015*, Jackie Kern and Jeffrey S. Vetter (Eds.). ACM, New York, NY, USA, 41:1–41:12. doi:10.1145/2807591.2807627
- [17] Carl Hewitt, Peter Bishop, and Richard Steiger. 1973. A universal modular ACTOR formalism for artificial intelligence. In *Proceedings of the 3rd International Joint Conference on Artificial Intelligence* (Stanford, USA) (IJCAI'73). Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 235–245.
- [18] Christian T. Jacobs, Satya P. Jammy, and Neil D. Sandham. 2017. OpenSBLI: A framework for the automated derivation and parallel execution of finite difference solvers on a range of computer architectures. *Journal of Computational Science* 18 (2017), 12–23. doi:10.1016/j.jocs.2016.11.001
- [19] Mathias Jacquelin, Mauricio Araya-Polo, and Jie Meng. [n. d.]. *Csl-Examples/Benchmarks/25-Pt-Stencil at Master · Cerebras/Csl-Examples*. <https://github.com/Cerebras/csl-examples/tree/master/benchmarks/25-pt-stencil>
- [20] Mathias Jacquelin, Mauricio Araya-Polo, and Jie Meng. 2022. Scalable Distributed High-Order Stencil Computations. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–13. doi:10.1109/SC41404.2022.00035
- [21] Sebastian Kuckuk and Harald Köstler. 2016. Automatic Generation of Massively Parallel Codes from ExaSlang. *Computation* 4, 3 (2016). doi:10.3390/computation4030027
- [22] Hsiang Tsung Kung and Charles E. Leiserson. 1979. Systolic arrays (for VLSI). In *Sparse Matrix Proceedings 1978*, Vol. 1. Society for industrial and applied mathematics Philadelphia, PA, USA, 256–282.
- [23] C. Lattner and V. Adve. 2004. LLVM: a compilation framework for lifelong program analysis & transformation. In *International Symposium on Code Generation and Optimization*, 2004. CGO 2004. 75–86. doi:10.1109/CGO.2004.1281665
- [24] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 2–14. doi:10.1109/CGO51591.2021.9370308
- [25] Christian Lengauer, Sven Apel, Matthias Bolten, Shigeru Chiba, Ulrich Rüde, Jürgen Teich, Armin Größlinger, Frank Hannig, Harald Köstler, Lisa Claus, Alexander Grebhahn, Stefan Groth, Stefan Kronawitter, Sebastian Kuckuk, Hannah Rittich, Christian Schmitt, and Jonas Schmitt. 2020. ExaStencils: Advanced Multigrid Solver Generation. In *Software for Exascale Computing - SPPEXA 2016-2019*, Hans-Joachim Bungartz, Severin Reiz, Benjamin Uekermann, Philipp Neumann, and Wolfgang E. Nagel (Eds.). Springer International Publishing, Cham, 405–452. https://doi.org/10.1007/978-3-030-47956-5_14
- [26] Sean Lie. 2023. Cerebras architecture deep dive: First look inside the hardware/software co-design for deep learning. *IEEE Micro* 43, 3 (2023), 18–30.
- [27] M. Louboutin, M. Lange, F. Luporini, N. Kukreja, P. A. Witte, F. J. Herrmann, P. Velesko, and G. J. Gorman. 2019. Devito (v3.1.0): an embedded domain-specific language for finite differences and geophysical exploration. *Geoscientific Model Development* 12, 3 (2019), 1165–1187. doi:10.5194/gmd-12-1165-2019
- [28] Hatem Ltaief, Yuxi Hong, Leighton Wilson, Mathias Jacquelin, Matteo Ravasi, and David Elliot Keyes. 2023. Scaling the “Memory Wall” for Multi-Dimensional Seismic Processing with Algebraic Compression on Cerebras CS-2 Systems. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Denver, CO, USA) (SC '23). Association for Computing Machinery,

- New York, NY, USA, Article 6, 12 pages. doi:10.1145/3581784.3627042
- [29] Fabio Luporini, Mathias Louboutin, Michael Lange, Navjot Kukreja, Philipp Witte, Jan Hückelheim, Charles Yount, Paul H. J. Kelly, Felix J. Herrmann, and Gerard J. Gorman. 2020. Architecture and Performance of Devito, a System for Automated Stencil Computation. *ACM Trans. Math. Softw.* 46, 1, Article 6 (Apr 2020), 28 pages. doi:10.1145/3374916
- [30] Thomas Melvin, Steve Mullerworth, Rupert Ford, Chris Maynard, and Mike Hobson. 2017. LFRic: Building a new Unified Model. In *EGU General Assembly Conference Abstracts*. 13021.
- [31] Aaron Meurer, Christopher P. Smith, Mateusz Paprocki, Ondřej Čertík, Sergey B. Kirpichev, Matthew Rocklin, AMiT Kumar, Sergiu Ivanov, Jason K. Moore, Sartaj Singh, Thilina Rathnayake, Sean Vig, Brian E. Granger, Richard P. Muller, Francesco Bonazzi, Harsh Gupta, Shivam Vats, Fredrik Johansson, Fabian Pedregosa, Matthew J. Curry, Andy R. Terrel, Štěpán Roučka, Ashutosh Saboo, Isuru Fernando, Sumith Kulal, Robert Cimrman, and Anthony Scopatz. 2017. SymPy: symbolic computing in Python. *PeerJ Computer Science* 3 (Jan. 2017), e103. doi:10.7717/peerj-cs.103
- [32] Marcelo Orenes-Vera, Ilya Sharapov, Robert Schreiber, Mathias Jacquelin, Philippe Vandermersch, and Sharan Chetlur. 2023. Wafer-scale fast fourier transforms. In *Proceedings of the 37th International Conference on Supercomputing*. 180–191.
- [33] Danny Perez, Aidan Thompson, Stan Moore, Tomas Oppelstrup, Ilya Sharapov, Kylee Santos, Amirali Sharifian, Delyan Z. Kalchev, Robert Schreiber, Scott Pakin, Edgar A. Leon, III Laros, James H., Michael James, and Sivasankaran Rajamanickam. 2025. Breaking the mold: Overcoming the time constraints of molecular dynamics on general-purpose hardware. *The Journal of Chemical Physics* 162, 7 (02 2025), 072501. doi:10.1063/5.0249193
- [34] A. R. Porter, J. Appleyard, M. Ashworth, R. W. Ford, J. Holt, H. Liu, and G. D. Riley. 2018. Portable multi- and many-core performance for finite-difference or finite-element codes – application to the free-surface component of NEMO (NEMOLite2D 1.0). *Geoscientific Model Development* 11, 8 (2018), 3447–3464. doi:10.5194/gmd-11-3447-2018
- [35] István Z. Reguly, Gihan R. Mudalige, and Michael B. Giles. 2018. Loop Tiling in Large-Scale Stencil Codes at Run-Time with OPS. *IEEE Trans. Parallel Distributed Syst.* 29, 4 (2018), 873–886. doi:10.1109/TPDS.2017.2778161
- [36] Kamil Rocki, Dirk Van Essendelft, Ilya Sharapov, Robert Schreiber, Michael Morrison, Vladimir Kibardin, Andrey Portnoy, Jean Francois Dietiker, Madhava Syamlal, and Michael James. 2020. Fast stencil-code computation on a wafer-scale processor. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Atlanta, Georgia) (SC '20). IEEE Press, Article 58, 14 pages.
- [37] Gabriel Rodriguez-Canal, Nick Brown, Maurice Jamieson, Emilien Bauer, Anton Lydike, and Tobias Grosser. 2023. Stencil-HMLS: A multi-layered approach to the automatic optimisation of stencil codes on FPGA. In *Proceedings of the SC'23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*. 556–565.
- [38] Ryuichi Sai, François P. Hamon, John Mellor-Crummey, and Mauricio Araya-Polo. 2024. Matrix-Free Finite Volume Kernels on a Dataflow Architecture. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis* (Atlanta, GA, USA) (SC '24). IEEE Press, Article 28, 11 pages. doi:10.1109/SC41406.2024.00034
- [39] Ryuichi Sai, Mathias Jacquelin, Francois Hamon, Mauricio Araya-Polo, and Randolph R. Settgaest. 2023. Massively Distributed Finite-Volume Flux Computation. In *Proceedings of the SC '23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis* (Denver, CO, USA) (SC-W '23). Association for Computing Machinery, New York, NY, USA, 1713–1720. doi:10.1145/3624062.3624252
- [40] Ryuichi Sai, John Mellor-Crummey, Jinfan Xu, and Mauricio Araya-Polo. 2024. Automated Code Generation of High-Order Stencils for a Dataflow Architecture. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis* (Atlanta, GA, USA) (SC '24). IEEE Press, Article 19, 13 pages. doi:10.1109/SC41406.2024.00025
- [41] Kylee Santos, Stan Moore, Tomas Oppelstrup, Amirali Sharifian, Ilya Sharapov, Aidan Thompson, Delyan Z. Kalchev, Danny Perez, Robert Schreiber, Scott Pakin, Edgar A. Leon, James H. Laros, Michael James, and Sivasankaran Rajamanickam. 2024. Breaking the Molecular Dynamics Timescale Barrier Using a Wafer-Scale System. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–13. doi:10.1109/SC41406.2024.00014
- [42] Tenstorrent. [n. d.]. *tt-mlir*. <https://github.com/tenstorrent/tt-mlir>
- [43] Felix Thaler, Stefan Moosbrugger, Carlos Osuna, Mauro Bianco, Hannes Vogt, Anton Afanasyev, Lukas Mosimann, Oliver Fuhrer, Thomas C. Schulthess, and Torsten Hoefler. 2019. Porting the COSMO Weather Model to Manycore CPUs. In *Proceedings of the Platform for Advanced Scientific Computing Conference* (Zurich, Switzerland) (PASC '19). Association for Computing Machinery, New York, NY, USA, Article 13, 11 pages. doi:10.1145/3324989.3325723
- [44] John Tramm, Bryce Allen, Kazutomo Yoshii, Andrew Siegel, and Leighton Wilson. 2024. Efficient algorithms for Monte Carlo particle transport on AI accelerator hardware. *Computer Physics Communications* 298 (2024), 109072. doi:10.1016/j.cpc.2023.109072
- [45] Anda Trifan, Defne Gorgun, Michael Salim, Zongyi Li, Alexander Brace, Maxim Zvyagin, Heng Ma, Austin Clyde, David Clark, David J Hardy, et al. 2022. Intelligent resolution: Integrating Cryo-EM with AI-driven multi-resolution simulations to observe the severe acute respiratory syndrome coronavirus-2 replication-transcription machinery in action. *The international journal of high performance computing applications* 36, 5-6 (2022), 603–623.
- [46] Yuxuan Wang, Cristian Tirelli, Lara Orlandic, Juan Sapriza, Rubén Rodríguez Álvarez, Giovanni Ansaloni, Laura Pozzi, and David Atienza. 2025. An mlir-based compilation framework for cgra application deployment. In *International Symposium on Applied Reconfigurable Computing*. Springer, 33–50.
- [47] Mino Woo, Terry Jordan, Robert Schreiber, Ilya Sharapov, Shaheer Muhammad, Abhishek Koneru, Michael James, and Dirk Van Essendelft. 2022. Disruptive Changes in Field Equation Modeling: A Simple Interface for Wafer Scale Engines. arXiv:2209.13768 [cs.DC] <https://arxiv.org/abs/2209.13768>
- [48] Edgar Zamora-Gómez, Pedro García-López, and Rubén Mondéjar. 2015. Continuation complexity: A callback hell for distributed systems. In *European Conference on Parallel Processing*. Springer, 286–298.
- [49] Maxim Zvyagin, Alexander Brace, Kyle Hippe, Yuntian Deng, Bin Zhang, Cindy Orozco Bohorquez, Austin Clyde, Bharat Kale, Danilo Perez-Rivera, Heng Ma, et al. 2023. GenSLMs: Genome-scale language models reveal SARS-CoV-2 evolutionary dynamics. *The International Journal of High Performance Computing Applications* 37, 6 (2023), 683–705.